

Assignment03 PCD part2A

Francesco Carlucci

francesco.carlucci6@studio.unibo.it

Marco Raggini

marco.raggini2@studio.unibo.it

Chapter 1

1.1 Analisi del problema

Il progetto consiste nel realizzare una versione cooperativa e distribuita del Sudoku con un'architettura decentralizzata basata su scambio di messaggi, in cui i giocatori possono creare nuove griglie, partecipare alla risoluzione di griglie create da altri, entrare ed uscire dinamicamente da qualsiasi griglia. Inoltre i giocatori devono poter vedere in modo consistente le celle della griglia selezionate dagli altri e i valori che vengono inseriti.

1.2 Strategia risolutiva

La strategia risolutiva adoperata sfrutta **MOM (Message Oriented Middleware)** per lo scambio di messaggi tra i giocatori. In particolare è stato utilizzato RabbitMQ come broker di messaggi che implementa questo modello.

1.2.1 Channels

Per prima cosa abbiamo creato quattro canali (channels) su cui i giocatori possono comunicare:

- `create_sudoku`: utilizzato per notificare della creazione di un nuovo Sudoku;
- `select_cell`: utilizzato per notificare la selezione di una cella da parte di un giocatore;
- `unselect_cell`: serve a segnalare la deselection di una cella precedentemente selezionata;

- `set_value`: serve a segnalare l’inserimento, la modifica o l’eliminazione di un valore di una cella.

1.2.2 Scambio dei messaggi

Ogni volta che un giocatore esegue un’azione (ad esempio crea una nuova griglia o seleziona una cella), viene generato un messaggio che contiene le informazioni necessarie per descrivere quell’evento. Il messaggio viene pubblicato sul relativo canale RabbitMQ, e da lì viene automaticamente distribuito a tutti i giocatori connessi al gioco.

1.2.3 Callback e aggiornamento dello stato

Ogni canale è associato a una funzione di *callback*, ovvero un metodo che viene eseguito quando arriva un nuovo messaggio su quel canale. Le callback si occupano di interpretare il contenuto del messaggio e di aggiornare di conseguenza lo stato locale del gioco. In questo modo, ogni giocatore mantiene una visione coerente e sincronizzata della griglia condivisa.

1.2.4 Gestione dei messaggi

Per semplificare la comunicazione tra i vari giocatori e garantire uno scambio di informazioni coerente, è stata introdotta la classe **MessageUtils**. Questa classe fornisce metodi dedicati alla serializzazione e deserializzazione dei messaggi, ossia alla conversione degli oggetti Java (come le griglie o le azioni dei giocatori) in stringhe testuali da inviare tramite RabbitMQ, e viceversa.

1.2.5 Architettura MVC

L’applicazione segue un’architettura di tipo **Model–View–Controller (MVC)**, che consente di separare in modo chiaro la logica di gioco, la gestione della comunicazione e l’interfaccia utente. Il **Model** è costituito dalla logica di funzionamento di ciascuna griglia di Sudoku e le informazioni sui giocatori; il **Controller** gestisce le interazioni dell’utente con la GUI e comunica alla view quando aggiornarsi; infine la **View** si occupa di aggiornare la rappresentazione grafica del gioco in base agli eventi notificati dal controller.